Technical Summary: Probabilistic Tree-of-thought Reasoning for Answering Knowledge-intensive Complex Questions

Kai-Yu Lu

1 Research Problem and Motivation

Knowledge-intensive complex question answering (Complex QA) requires reasoning over multiple facts that are not reliably stored or up-to-date in a large language model (LLM). Chain-of-thought (CoT) prompting improves multi-step reasoning but often produces factually incorrect intermediate steps when required knowledge is missing from model parameters. Retrieval-augmented reasoning addresses this limitation by conditioning CoT on external documents, but prior chain-based retrieval methods remain insufficient for Complex QA due to two empirically emphasized failure modes.

First, *negative retrieval* occurs when retrieved paragraphs are unnecessary or incorrect, which misleads downstream reasoning and causes wrong answers. Second, *limited sight* arises from strictly sequential reasoning, in which a local error in one step propagates forward and cannot be globally revised. Complex QA therefore requires a framework that (i) selectively benefits from external retrieval without being dominated by distractors, and (ii) supports global reconsideration of intermediate decisions rather than committing to a single chain.

2 Related Work

Retrieval-augmented LLMs can be grouped by retrieval frequency. One-time retrieval methods retrieve documents once using the original question, which is insufficient for multi-hop information needs. Multi-time retrieval methods interleave generation and retrieval, for example by triggering retrieval at each sub-question (Self-Ask) or at each CoT sentence (IRCoT), and by iterative retrieval-generation loops (ITER-RETGEN). These methods improve factuality but remain chain-based, which directly exposes them to negative retrieval and limited sight. ReAct integrates reasoning and tool use, and related contemporaneous approaches explore meta-reasoning or iterative refinement, but differences in evaluation settings often limit direct comparability.

Separately, tree-structured reasoning has been explored for complex computation graphs and structured question answering. Prior tree-like approaches often depend on supervised sub-question annotations or specialized trained modules (such as semantic parsers, readers, and link predictors). The present work emphasizes a fully LLM-driven pipeline that constructs a hierarchical query tree via prompting and performs probabilistic reasoning over the tree while explicitly accounting for uncertainty.

3 Dataset Construction

No new dataset is constructed. Evaluation uses three public Complex QA benchmarks in an open-domain setting: HotpotQA, MuSiQue, and 2WikiMultiHopQA (2WikiMQA). Retrieval corpora follow the IRCoT setting: HotpotQA uses an October 2017 Wikipedia dump, while MuSiQue and 2WikiMQA use corpora

Table 1: Datasets and retrieval corpora in the open-domain evaluation setting.

Item	Details
Datasets	HotpotQA, MuSiQue, 2WikiMultiHopQA (2WikiMQA).
Task type	Open-domain knowledge-intensive complex question answering with multi-hop reasoning.
Retrieval corpus (HotpotQA)	October 2017 Wikipedia dump.
Retrieval corpus (MuSiQue, 2WikiMQA)	Combined supporting and non-supporting paragraphs for all questions in train, development, and test sets.
Development set	100 questions sampled from the original development set (per dataset).
Test set	500 questions sampled from the original development set (per dataset).
Preprocessing	Not specified in the paper.
Training data usage	Not specified in the paper (the instantiated framework uses prompting with an external retriever and an LLM backend).

constructed by combining supporting and non-supporting paragraphs across train, development, and test splits.

The evaluation splits follow the IRCoT-provided sampling procedure: for each dataset, 100 questions are sampled from the original development set as a development set and 500 as a test set. Dataset scale details beyond these sampled split sizes (such as full train size used by the original dataset creators) are not specified in the paper.

4 Query Protocol and Task Definitions

4.1 Task definition and success criterion

Given a natural-language complex question, the system outputs a final answer string. Success is measured by **Answer F1**, which quantifies token overlap between prediction and gold answer by balancing precision and recall. Higher F1 indicates that the predicted answer matches the reference more accurately while maintaining coverage of the correct content.

4.2 Query tree formalization

A complex question is translated into a **query tree** T. The root node is the original question, and each non-root node is a sub-question of its parent. Leaf nodes are **atomic questions** that are not further decomposed. Nodes can contain **reference tokens** (for example, "#1") that refer to answers of earlier sibling questions, enabling entity linking across sub-questions. The tree is indexed in breadth-first order for generation, while solving proceeds from leaves to root using post-order traversal.

4.3 Minimal illustrative protocol example

Consider a question that implicitly requires at least three facts: (i) identify an entity, (ii) identify a related entity, and (iii) query a temporal relation between them. The query tree produces leaf questions that retrieve or recall these atomic facts. During reasoning, each leaf is answered by both open-book QA (with retrieved paragraphs) and closed-book QA (without external context), and a confidence-guided selection chooses the

more reliable leaf answer. Parent nodes then aggregate child question-answer pairs and re-reason over them, allowing global correction when a child sub-question or retrieval is misleading.

5 Modeling Approach

The framework disentangles Complex QA into two stages: **understanding** and **reasoning**.

5.1 Stage 1: Understanding via hierarchical decomposition

An LLM generates a hierarchical question decomposition tree in a JSON format through few-shot prompting. For each non-leaf node q_i, the framework computes a **decomposition score** that estimates confidence that q_i can be decomposed into its children. The score is defined as the average log-likelihood of the serialized child-question sequence.

$$ds_i \;=\; \frac{1}{|seq_i|} \sum_{j=1}^{|seq_i|} \log p\big(x_j \mid x_{<j}, [Q, T_i^{\text{before}}, q_i]\big) \tag{1}$$

Purpose. Quantify confidence in the generated decomposition at node q_i during the understanding stage, to be used later in probabilistic reasoning.
Symbols. ds_i is the decomposition score for node q_i. $seq_i = \langle x_1, \ldots, x_{|seq_i|} \rangle$ is the serialized sequence of child questions for q_i. x_j is the j-th token in seq_i. $x_{<j}$ denotes the prefix tokens before position j. Q is the original input question (root). T_i^{before} is the partial tree generated before expanding node q_i under breadth-first generation. $p(\cdot)$ is the next-token probability from the LLM. $\log$ is the natural logarithm. $[\cdot]$ denotes concatenation in the specified order.
Intuition. High-quality decompositions are assigned higher generation probability by an LLM; low probability indicates that the child-question sequence is unlikely under the model and therefore less trustworthy.
Usage. ds_i is computed for each non-leaf node after the query tree is generated, and later contributes to the confidence of child-aggregation reasoning for that node.

5.2 Stage 2: Probabilistic reasoning over the tree

Reasoning proceeds from leaf to root. For each node, three QA modules produce candidate answers and confidence scores, followed by selecting the most confident candidate.

- **Closed-book QA (CB).** Answers using only parametric knowledge, with empty external context.
- **Open-book QA (OB).** Answers conditioned on retrieved paragraphs from an external corpus. For non-leaf nodes, the context includes retrieved paragraphs for the node and for its descendants, which supports evidence sharing across the subtree.
- **Child-aggregating QA (CA).** Answers by packing child question-answer pairs as the context and performing CoT reasoning to produce a parent answer, enabling global meta-reasoning over sub-results.

5.3 Confidence from explanation likelihood

The central mechanism is a confidence estimator that scores a candidate by the likelihood of its CoT explanation. Let $e = \langle x_1, \ldots, x_{|e|} \rangle$ denote the generated explanation tokens for CA under context c. The intermediate confidence is defined as the average log-likelihood of e.

$$\tilde{s}_i^{\text{ca}} \;=\; \frac{1}{|e|}\sum_{j=1}^{|e|}\log p(x_j \mid x_{<j}, [c, q_i]) \tag{2}$$

Purpose. Estimate how confident the LLM is in the derived answer by scoring the explanation that supports it.
Symbols. $\tilde{s}_i^{\text{ca}}$ is the explanation-likelihood confidence for child-aggregating QA at node q_i. e is the generated explanation sequence for answering q_i under CA. x_j and $x_{<j}$ are explanation tokens and their prefix. c is the context, consisting of child question-answer pairs for CA. $p(\cdot)$ is the LLM next-token probability. $[c, q_i]$ is the concatenated prompt with context and question.
Intuition. If the model assigns high probability to a coherent explanation, the derived answer is treated as more reliable; low explanation likelihood indicates uncertainty or inconsistency.
Usage. $\tilde{s}_i^{\text{ca}}$ is computed whenever CA is executed at node q_i and is combined with decomposition and child confidences to form the final CA confidence.

5.4 Confidence integration for child aggregation

Since CA depends on the correctness of decomposition and child answers, the final CA confidence combines decomposition score, children confidence scores, and explanation likelihood.

$$s_i^{\text{ca}} \;=\; \frac{1}{n_i + 2}\left(ds_i + \sum_{j \in \text{children}(i)} s_j + \tilde{s}_i^{\text{ca}}\right) \tag{3}$$

Purpose. Integrate uncertainty sources to obtain a calibrated confidence for CA at node q_i.
Symbols. s_i^{ca} is the final CA confidence at node q_i. n_i is the number of children of node q_i. ds_i is the decomposition score from Equation 1. children(i) is the set of child node indices for q_i. s_j is the selected confidence score for child node q_j after resolving that node. $\tilde{s}_i^{\text{ca}}$ is the explanation-likelihood confidence from Equation 2.
Intuition. A parent answer is reliable when the decomposition is reliable, the child answers are reliable, and the parent-level explanation is internally consistent under the aggregated context.
Usage. s_i^{ca} is computed at each non-leaf node and compared against OB and CB confidences; the candidate with maximum confidence is selected as the node solution before propagating upward.

5.5 Key design consequences

Negative retrieval mitigation. At leaf nodes, CB and OB are both executed and the more confident answer is selected, preventing automatic commitment to retrieved content when retrieval is misleading.
Limited sight mitigation. Tree structure enables CA at non-leaf nodes, which re-reasons globally over child results and can correct errors originating from flawed retrieval or flawed child sub-questions.

6 Empirical Results

6.1 Main results on three Complex QA datasets

Table 3 reports Answer F1. Compared with IRCoT under comparable settings, ProbTree improves HotpotQA overall F1 from 60.2 to 62.6 (absolute +2.4), and improves MuSiQue overall F1 from 34.2 to 41.5 (absolute +7.3). On 2WikiMQA, ProbTree improves over IRCoT from 63.8 to 71.8 (absolute +8.0) and

Table 2: Implementation and evaluation settings.

Component	Setting
LLM backend	OpenAI GPT3 (text-davinci-003).
Decoding temperature	0 for stable output.
Retriever	BM25 implemented with Elasticsearch.
Retrieval per node	Retrieve top-K paragraphs for each node query; $K \in \{3, 5, 7\}$ selected on the development set.
Open-domain evaluation	Retrieval is performed against the specified corpus; answers are evaluated by Answer F1.
Prompt exemplars	Not specified in the main text; appendix describes manually prepared exemplars for several prompts.
Training procedure	Not specified in the paper (framework instantiation relies on prompting and retrieval, not gradient training).
Hardware and cost	Not specified in the paper.

Table 3: Answer F1 (%) results under the open-domain setting.

Method	HotpotQA	MuSiQue	2WikiMQA
Direct (no retrieval)	39.5	16.9	33.6
CoT (no retrieval)	46.7	23.1	39.4
One-step retrieval (OneR)	53.2	25.7	48.1
IRCoT	60.2	34.2	63.8
Self-Ask	49.4	33.4	66.6
ProbTree	62.6	41.5	71.8
ProbTree + Google snippet for leaf nodes	64.1	Not specified in the paper.	Not specified in the paper.

improves over Self-Ask from 66.6 to 71.8 (absolute +5.2). These gains indicate that the tree structure and confidence-based integration provide advantages beyond chain-based retrieval augmentation.

Performance advantages are most pronounced on more challenging compositional subsets. On MuSiQue, ProbTree achieves 38.1 on 3-hop questions and 23.3 on 4-hop questions, exceeding IRCoT (26.3 and 20.1) and reflecting improved robustness as composition complexity increases. On 2WikiMQA, ProbTree improves the Inference subset from 45.4 (IRCoT) to 68.7 and the Bridge-Comparison subset from 79.0 (IRCoT) to 95.2, indicating large gains on implicit reasoning and combined bridge-and-compare structures.

A reported variant that adds a top-1 Google Search snippet for each leaf question improves HotpotQA from 62.6 to 64.1, suggesting that stronger retrieval sub-modules further increase end-to-end performance.

6.2 Ablation study: effect of tree reasoning and probabilistic selection

Table 4 isolates key components. Replacing tree reasoning with sequential decomposition (SD) drops HotpotQA from 64.1 to 51.7 (absolute −12.4), demonstrating that global reconsideration enabled by the tree is critical. Removing child-aggregating QA (w/o CA) yields minimal drop on HotpotQA (−0.2) but large drops on MuSiQue and 2WikiMQA (both −6.7), indicating that CA is particularly important for harder compositional reasoning.

The confidence mechanism is validated by comparing against random choice (w/ RC), which collapses performance (HotpotQA 53.2, MuSiQue 25.0, 2WikiMQA 52.0), and against self-consistency variants (SC@3, SC@5), which underperform ProbTree while requiring substantially more decoding. Removing

Table 4: Ablation results (Answer F1, %).

Variant	HotpotQA	MuSiQue	2WikiMQA
ProbTree (with Google snippet for HotpotQA leaf nodes)	64.1	41.5	71.8
Sequential decomposition (SD)	51.7	39.0	69.2
w/o Child-aggregating QA (w/o CA)	63.9	34.8	65.1
Random choosing (w/ RC)	53.2	25.0	52.0
Self-consistency (w/ SC@3)	62.4	38.4	68.8
Self-consistency (w/ SC@5)	63.0	40.7	70.1
w/o Closed-book QA (w/o CB)	63.6	38.4	70.4
w/o Open-book QA (w/o OB)	46.3	23.2	42.3

Table 5: Error category proportions (%) among 50 analyzed errors per dataset.

Error type	HotpotQA	MuSiQue	2WikiMQA
Evaluation limitation	38	46	46
Inaccurate data annotation	16	16	8
Retrieval error	8	12	14
Reasoning error	6	8	14
Decomposition error	16	12	8
Inaccurate confidence	16	6	10

open-book QA (w/o OB) causes severe degradation across all datasets, confirming that external evidence is indispensable in this setting. Removing closed-book QA (w/o CB) causes smaller but consistent drops, indicating that parametric knowledge remains complementary even with retrieval.

6.3 Additional ablations on uncertainty and evidence sharing

Ignoring the decomposition score (ds_i) in the CA confidence computation slightly reduces performance (HotpotQA 63.9, MuSiQue 41.3, 2WikiMQA 71.6), indicating that decomposition uncertainty contributes incremental gains. Removing retrieved paragraphs from descendants in open-book QA reduces HotpotQA from 64.1 to 62.3 and MuSiQue from 41.5 to 39.7, indicating that subtree evidence sharing improves robustness for non-leaf reasoning.

6.4 Error analysis

Errors are grouped into six categories: evaluation limitation, inaccurate data annotation, retrieval error, reasoning error, decomposition error, and inaccurate confidence. Decomposition error is emphasized as most challenging in HotpotQA due to complex syntactic structures. Retrieval, reasoning, and decomposition errors each exceed 30% on every dataset, while inaccurate confidence constitutes a smaller fraction, indicating that remaining limitations are dominated by retrieval quality and reasoning reliability rather than confidence estimation alone.

7 Summary

Probabilistic tree-of-thought reasoning addresses knowledge-intensive Complex QA by replacing chain-based retrieval reasoning with a query tree and performing confidence-aware reasoning from leaves to root. The approach combines three complementary QA modules: open-book QA for external evidence, closed-book QA for parametric knowledge, and child-aggregating QA for global meta-reasoning over sub-answers. Uncertainty is explicitly modeled through decomposition likelihood and explanation-likelihood confidence, enabling selection of reliable answers and reducing negative retrieval effects. Empirically, the method achieves consistent gains over strong chain-based baselines on HotpotQA, MuSiQue, and 2WikiMQA, with particularly large improvements on harder compositional subsets. Limitations are explicitly tied to backend LLM requirements (few-shot CoT ability, calibration, and long context), additional computational cost from multiple QA module executions per node, and restriction to unstructured document corpora as the external knowledge source.